Project Proposal: A Compiler from StragglAR Schedules to NCCL Send/Receive Programs

Team 31 (Chesterjesters): Wyatt Sell, David Valarezo, Nathnael Tesfaw, Aneesh Naresh
Advisor: Arjun Devraj · Compute: 2 Perlmutter nodes (8×A100, NCCL over Slingshot)

Goal. StragglAR's synthesizer outputs AllReduce schedules (which rank sends which chunk to whom, per step) in its own text format, but turning a schedule into runnable NCCL code is done by hand. We will build a compiler that parses the schedule into a documented intermediate representation (IR) and generates a C++/CUDA program of grouped `ncclSend`/`ncclRecv` calls with local reductions. Success means the generated program runs without deadlock on 8 GPUs across two nodes, gives the same output as `ncclAllReduce`, and, with the same straggler delay injected into both, finishes faster than `ncclAllReduce`.

Pipeline. StragglAR schedule (text) → parser → IR (JSON) → code generator → NCCL program → test harness on Perlmutter.

Avoiding blocking. (1) The IR format is fixed by Oct 29, so the parser and code generator are built against it in parallel. (2) Wyatt hand-writes an n=4 IR file on Oct 29 so David never waits on the parser. (3) The test harness calls any AllReduce through one function interface, so Aneesh tests NCCL's built-in AllReduce first and swaps in generated code later. (4) Nathnael's launch script is ready by Nov 1, so everyone can run on Perlmutter early.

Who does what. Each person owns one stage end to end, presents it at the midpoint, and makes its poster panel.

Member	Owns	Poster panel
Wyatt Sell	Running StragglAR for $n = 2, 4, 8, 16$; schedule parser; IR design and its written spec; IR simulator that checks a schedule produces a correct AllReduce (CPU only, no GPUs).	Problem, StragglAR background, IR design.
David Valarezo	Code generator: IR → C++/CUDA with one `ncclGroupStart/End` block per step; local reduction kernel; deadlock-free ordering.	Compiler design and a generated-code example.
Nathnael Tesfaw	Perlmutter setup: NCCL build, Makefile, Slurm script for 1 and 2 nodes; a hand-written send/recv test on 8 GPUs; runs all multi-node jobs.	Experimental setup and system details.
Aneesh Naresh	Test harness: buffer init, comparison against `ncclAllReduce`, straggler delay injection on rank $n-1$, timing ours vs. `ncclAllReduce` with the same delay on both; result plots.	Results: correctness and speedup vs. `ncclAllReduce` under a straggler.

Timeline.

Dates	Work
Oct 26 – Oct 29	All: kickoff, read StragglAR code, agree on and freeze the IR format. Wyatt runs StragglAR and writes the sample n=4 IR. Nathnael gets Perlmutter access for all four members.
Oct 30 – Nov 8	In parallel: Wyatt builds parser + simulator; David generates code from the sample IR; Nathnael delivers the launch script (Nov 1) and the 8-GPU send/recv test; Aneesh builds the harness against `ncclAllReduce`.
Nov 9	**Midpoint presentation**: schedules for several n, the IR, generated code for n=4 running on one node and passing the harness. Each member presents their stage.
Nov 10 – Nov 22	Full pipeline for n=8 on two nodes. Wyatt and David connect parser to generator; Nathnael and Aneesh run and debug on 8 GPUs. Goal: no deadlock and correct results by Nov 22.
Nov 23 – Dec 1	Aneesh and Nathnael time the generated program against `ncclAllReduce` with the same straggler delay on both (sweeping delay × buffer size). If we are not faster, Wyatt and David profile and tune the generated code; otherwise they finish the IR docs. Thanksgiving week is slack.
Dec 2 – Dec 6	Each member makes their poster panel; Dec 5 full team review and practice.
Dec 7	**Final poster session.**

Deliverables. Schedules for several power-of-two n; the compiler with a documented IR; a test showing the generated program runs without deadlock on 8 GPUs across two nodes, matches `ncclAllReduce`, and beats it when the same straggler delay is injected into both.